FIAP – Faculdade de Informática e Administração Paulista
Análise e Desenvolvimento de Sistemas

EcoRank

API Restful de ranking de ações sustentáveis

Domain Driven Design using Java – Entrega Final

Nome da solução: EcoRank
Nome da equipe: Inovatech

Integrantes

Nome	RM
Guilherme Joel de Camargo	RM570403
Pedro Felipe de Castro Rosa	RM569049
Arthur Chang Skolimoski	RM572510
Victor Ferreira Gomes	RM569273
Rafaela Donadio Figueiredo Tavares	RM572797
Pedro Henrique Cardoso Cavalcante de Souza	RM570464

Professor: Eduardo Gondo
São Paulo – 2026

Sumário

1. Objetivo e escopo do projeto
2. Breve descrição das principais funcionalidades
3. Arquitetura e tecnologias utilizadas
4. Tabela de endpoints (API Restful)
5. Exemplos de requisição e resposta
6. Protótipo – prints das telas implementadas
7. Modelo de Entidade-Relacionamento (MER)
8. Diagrama de classes atualizado
9. Procedimentos para rodar a aplicação
10. Deploy da aplicação na nuvem (Render)
11. Links da entrega

1. Objetivo e escopo do projeto

O EcoRank é a solução de gamificação da plataforma SoulUp, criada para incentivar hábitos sustentáveis. O usuário cria uma conta, registra ações sustentáveis (reciclagem, mobilidade, economia de água e de energia, plantio, entre outras) anexando uma foto ou um vídeo como comprovação e, a cada ação válida, ganha pontos, sobe de nível, avança em desafios e missões, desbloqueia conquistas e disputa posições em um ranking.

O objetivo desta entrega é disponibilizar uma API Restful com Spring Boot que serve como back-end do front-end em React, substituindo os dados de demonstração por dados persistidos em banco Oracle. A aplicação segue a arquitetura em camadas vista em aula (Resource, Service/BO e Repository/DAO), usa JDBC com DataSource e está publicada na nuvem por meio de Docker no Render.

Escopo implementado

- Cadastro, login, logout e sessão por token; perfil do próprio usuário;
- Registro de ações com foto/vídeo, cálculo de pontos no servidor e histórico;
- Níveis, ranking, dashboard e indicadores de impacto ambiental (estimativa de CO_2);
- Desafios com progresso individual, objetivos pessoais, missões e conquistas;
- Formulário de contato e conteúdos/notícias ambientais;
- CRUD completo de usuários, ações, níveis, missões, conquistas, objetivos, desafios, itens do catálogo, conteúdos e mensagens de contato;
- Perfis USUARIO e ADMIN, tratamento de exceções, CORS restrito e integração com o front-end.

Fora do escopo

Eventos, recompensas e favoritos aparecem na navegação do front-end, mas continuam apenas demonstrativos, sem back-end. A preferência de tema (claro/escuro) é visual e fica salva no navegador.

2. Breve descrição das principais funcionalidades

Funcionalidade	Descrição
Cadastro e login	Conta com nome, e-mail único e senha (6 a 100 caracteres). A senha é guardada com hash PBKDF2 e salt. O login devolve um token de sessão (validade de 7 dias) usado no cabeçalho Authorization. Após 5 senhas erradas, o e-mail fica bloqueado por 15 minutos.
Registro de ações	O usuário escolhe uma ação do catálogo, descreve (10 a 180 caracteres) e anexa foto (JPG, PNG, WEBP ou GIF, até 10 MB) ou vídeo (MP4 ou MOV, até 25 MB e 45 segundos). O servidor confere o tipo real do arquivo, a duração do vídeo e calcula os pontos pelo catálogo. A comprovação é gravada no Oracle.
Pontos e níveis	Pontos e nível são calculados somente no servidor: Bronze (0), Prata (200), Ouro (400), Diamante (700) e Esmeralda (1000 pontos), conforme a tabela NIVEL.
Ranking	Lista os usuários da maior para a menor pontuação, sem expor o e-mail. Em caso de empate, fica na frente quem se cadastrou primeiro.
Dashboard e impacto	Resumo com pontos, ações, nível, pontos para o próximo nível, posição, indicadores da semana, distribuição por categoria e evolução dos últimos 30 dias. O CO_2 é uma estimativa por tipo de ação (fator em kg por ação do catálogo) e é apresentado como valor estimado.
Desafios	O usuário participa de um desafio e o progresso é calculado a partir das ações aprovadas depois da participação. Ao atingir a meta, a recompensa é creditada uma única vez e uma conquista é desbloqueada.
Objetivos pessoais	Cada usuário cria, edita, atualiza o progresso e exclui apenas os próprios objetivos.
Missões e conquistas	Cada ação aprovada soma +1 nas missões ativas do usuário; ao atingir o objetivo, a recompensa em pontos é creditada. A primeira ação aprovada desbloqueia a conquista “Primeira Ação”.
Contato e conteúdos	O “Fale conosco” grava a mensagem (limite de 3 por hora por e-mail). A equipe (ADMIN) acompanha o status. As notícias ambientais são consultadas com filtro por categoria e busca.
Administração	Usuários com perfil ADMIN mantêm níveis, catálogo de ações, desafios, missões, conquistas, conteúdos e mensagens de contato.

Principais regras de negócio e validações

- Campos obrigatórios, tamanhos máximos e e-mail válido e não repetido;
- Existência do usuário antes de criar ações, missões ou conquistas;
- Pontos definidos no servidor, nunca informados pelo navegador;
- Ações aprovadas não podem ser alteradas; ao excluir uma ação aprovada, os pontos dela são retirados;
- Cada usuário só acessa os próprios dados; dados administrativos exigem o perfil ADMIN.

Quando uma regra é violada, a API devolve o status HTTP adequado (400, 401, 403, 404 ou 429) com uma mensagem em JSON no formato {"mensagem": "..."}.

3. Arquitetura e tecnologias utilizadas

A aplicação é organizada em camadas, cada uma com uma responsabilidade:

Pacote	Responsabilidade	No enunciado
`model`	Classes que representam as tabelas do banco (Usuario, AcaoSustentavel, Missao, Conquista, Nivel, Desafio, Objetivo etc.) e o enum Status.	Camada modelo
`repository`	Acesso ao banco com JDBC: DataSource injetado, PreparedStatement e o CRUD de cada tabela. O DataSource (pool de conexões HikariCP) é configurado por variáveis de ambiente.	Camada DAO / conexão

Pacote	Responsabilidade	No enunciado
`service`	Regras de negócio, validações, cálculo de pontos, níveis, desafios, autenticação e permissões. Usa @Transactional ao gravar.	Camada BO
`resource`	Classes @RestController com os endpoints, verbos HTTP e códigos de status.	Camada Resource
`dto e mapper`	Records DTO que trafegam na API e Mappers que convertem DTO e entidade (a entidade nunca é exposta).	—
`exception`	EcoRankException (regra de negócio, 400) e suas especializações: RecursoNaoEncontrado (404), NaoAutorizado (401), AcessoNegado (403) e MuitasTentativas (429).	Tratamento de exceções
`config`	CorsFilter: libera o acesso do front-end somente para as origens autorizadas.	CorsFilter

Tecnologias: Java 21, Spring Boot 4.1.0 (starters webmvc e jdbc), Maven, driver Oracle (ojdbc11), banco de dados Oracle, Docker e Render. Front-end: React com TypeScript e Vite.

Fluxo de uma requisição: Cliente HTTP (React) → Resource → Service (BO) → Repository (DAO) → Oracle, e o caminho inverso com DTO em JSON.

Segurança: senhas com PBKDF2-HMAC-SHA256 (310.000 iterações) e salt; token de sessão aleatório de 256 bits com validade de 7 dias e revogação no logout; perfis USUARIO e ADMIN; credenciais do banco somente em variáveis de ambiente; CORS restrito às origens autorizadas; limite de tentativas de login e de mensagens de contato.

4. Tabela de endpoints (API Restful)

URL base local: http://localhost:8080 (em produção, a URL do Render). Todos os dados trafegam em JSON, exceto o envio de ações com comprovante (multipart/form-data). Rotas que não são públicas exigem o cabeçalho Authorization: Bearer <token>.

Códigos de status: 200 OK · 201 Created · 204 No Content · 400 Bad Request (validação ou regra de negócio) · 401 Unauthorized (sem login, token inválido ou expirado) · 403 Forbidden (sem permissão) · 404 Not Found · 429 Too Many Requests (limite de tentativas) · 500 Internal Server Error.

Acesso: Público = sem login; Login = qualquer usuário logado, sempre sobre os próprios dados; Dono ou ADMIN = o próprio usuário ou um administrador; ADMIN = somente administradores.

Verbo	URI	Descrição	Acesso	Sucesso	Erros possíveis
Autenticação e perfil					
POST	`/api/v1/auth/cadastro`	Cria a conta e já inicia a sessão (devolve o token)	Público	**201**	400, 500
POST	`/api/v1/auth/login`	Autentica e devolve o token de sessão	Público	**200**	401, 429, 500
POST	`/api/v1/auth/logout`	Encerra a sessão (invalida o token)	Login	**204**	401, 500
GET	`/api/v1/usuarios/me`	Dados do usuário logado	Login	**200**	401, 500
PUT	`/api/v1/usuarios/me`	Atualiza nome e e-mail do próprio perfil	Login	**200**	400, 401, 500
Usuários					
GET	`/api/v1/usuarios`	Lista todos os usuários	ADMIN	**200**	401, 403, 500
GET	`/api/v1/usuarios/{id}`	Busca um usuário pelo id	Dono ou ADMIN	**200**	401, 403, 404, 500
POST	`/api/v1/usuarios`	Cadastra um usuário (sem senha)	ADMIN	**201**	400, 401, 403, 500
PUT	`/api/v1/usuarios/{id}`	Atualiza nome e e-mail do usuário	Dono ou ADMIN	**200**	400, 401, 403, 404, 500
DELETE	`/api/v1/usuarios/{id}`	Exclui o usuário e seus dados	Dono ou ADMIN	**204**	401, 403, 404, 500
EcoRank (dashboard e impacto)					
GET	`/api/v1/ecorank/me`	Resumo do dashboard: pontos, nível, posição, semana, CO_2, categorias	Login	**200**	401, 500
GET	`/api/v1/impacto/me`	Evolução de pontos (30 dias), CO_2 estimado e metodologia	Login	**200**	401, 500
Ranking e níveis					
GET	`/api/v1/niveis`	Lista os níveis e pontos necessários	Público	**200**	500
GET	`/api/v1/niveis/{id}`	Busca um nível pelo id	Público	**200**	404, 500
POST	`/api/v1/niveis`	Cadastra um nível	ADMIN	**201**	400, 401, 403, 500
PUT	`/api/v1/niveis/{id}`	Atualiza um nível	ADMIN	**200**	400, 401, 403, 404, 500
DELETE	`/api/v1/niveis/{id}`	Exclui um nível	ADMIN	**204**	401, 403, 404, 500
GET	`/api/v1/ranking`	Ranking geral por pontos (sem e-mail)	Público	**200**	500

Verbo	URI	Descrição	Acesso	Sucesso	Erros possíveis
Ações e comprovantes					
GET	`/api/v1/acoes/catalogo`	Catálogo oficial de ações (rótulo, categoria, pontos, CO_2)	Público	**200**	500
GET	`/api/v1/acoes/catalogo/{id}`	Busca um item do catálogo pelo id	Público	**200**	404, 500
POST	`/api/v1/acoes/catalogo`	Cadastra um item no catálogo	ADMIN	**201**	400, 401, 403, 500
PUT	`/api/v1/acoes/catalogo/{id}`	Atualiza um item do catálogo	ADMIN	**200**	400, 401, 403, 404, 500
DELETE	`/api/v1/acoes/catalogo/{id}`	Exclui um item do catálogo	ADMIN	**204**	401, 403, 404, 500
GET	`/api/v1/acoes/me`	Histórico de ações do usuário logado	Login	**200**	401, 500
POST	`/api/v1/acoes/enviar`	Envia ação com foto/vídeo (multipart); o servidor valida e calcula os pontos	Login	**201**	400, 401, 404, 500
GET	`/api/v1/acoes`	Lista todas as ações	ADMIN	**200**	401, 403, 500
GET	`/api/v1/acoes/usuario/{idUsuario}`	Ações de um usuário	Dono ou ADMIN	**200**	401, 403, 404, 500
GET	`/api/v1/acoes/{id}`	Busca uma ação pelo id	Dono ou ADMIN	**200**	401, 403, 404, 500
POST	`/api/v1/acoes`	Registra ação por link de evidência (pontua pela categoria)	ADMIN	**201**	400, 401, 403, 500
PUT	`/api/v1/acoes/{id}`	Corrige e reenvia uma ação rejeitada	ADMIN	**200**	400, 401, 403, 404, 500
DELETE	`/api/v1/acoes/{id}`	Exclui a ação (retira os pontos se estava aprovada)	Dono ou ADMIN	**204**	401, 403, 404, 500
GET	`/api/v1/comprovacoes/{arquivo}`	Baixa a foto/vídeo de comprovação	Dono da ação	**200**	401, 404, 500
Desafios					
GET	`/api/v1/desafios`	Catálogo de desafios	Público	**200**	500
GET	`/api/v1/desafios/me`	Desafios com o progresso do usuário	Login	**200**	401, 500
POST	`/api/v1/desafios/{id}/participar`	Entra em um desafio	Login	**201**	400, 401, 404, 500
GET	`/api/v1/desafios/{id}`	Busca um desafio pelo id	Público	**200**	404, 500
POST	`/api/v1/desafios`	Cadastra um desafio	ADMIN	**201**	400, 401, 403, 500
PUT	`/api/v1/desafios/{id}`	Atualiza um desafio	ADMIN	**200**	400, 401, 403, 404, 500
DELETE	`/api/v1/desafios/{id}`	Exclui um desafio	ADMIN	**204**	401, 403, 404, 500
Objetivos					
GET	`/api/v1/objetivos`	Lista os objetivos do usuário logado	Login	**200**	401, 500
POST	`/api/v1/objetivos`	Cria um objetivo pessoal	Login	**201**	400, 401, 500
PUT	`/api/v1/objetivos/{id}`	Atualiza objetivo e progresso (somente o dono)	Login	**200**	400, 401, 404, 500
DELETE	`/api/v1/objetivos/{id}`	Exclui um objetivo (somente o dono)	Login	**204**	401, 404, 500
Missões					
GET	`/api/v1/missoes`	Lista todas as missões	ADMIN	**200**	401, 403, 500
GET	`/api/v1/missoes/usuario/{idUsuario}`	Missões de um usuário	Dono ou ADMIN	**200**	401, 403, 404, 500

Verbo	URI	Descrição	Acesso	Sucesso	Erros possíveis
GET	`/api/v1/missoes/{id}`	Busca uma missão pelo id	Dono ou ADMIN	**200**	401, 403, 404, 500
POST	`/api/v1/missoes`	Cadastra uma missão	ADMIN	**201**	400, 401, 403, 500
PUT	`/api/v1/missoes/{id}`	Atualiza uma missão	ADMIN	**200**	400, 401, 403, 404, 500
DELETE	`/api/v1/missoes/{id}`	Exclui uma missão	ADMIN	**204**	401, 403, 404, 500
Conquistas					
GET	`/api/v1/conquistas`	Lista todas as conquistas	ADMIN	**200**	401, 403, 500
GET	`/api/v1/conquistas/usuario/{idUsuario}`	Conquistas de um usuário	Dono ou ADMIN	**200**	401, 403, 404, 500
GET	`/api/v1/conquistas/{id}`	Busca uma conquista pelo id	Dono ou ADMIN	**200**	401, 403, 404, 500
POST	`/api/v1/conquistas`	Cadastra uma conquista	ADMIN	**201**	400, 401, 403, 500
PUT	`/api/v1/conquistas/{id}`	Atualiza uma conquista	ADMIN	**200**	400, 401, 403, 404, 500
DELETE	`/api/v1/conquistas/{id}`	Exclui uma conquista	ADMIN	**204**	401, 403, 404, 500
Contato					
POST	`/api/v1/contato`	Envia mensagem do “Fale conosco” (limite por hora)	Público	**201**	400, 429, 500
GET	`/api/v1/contato`	Lista as mensagens recebidas	ADMIN	**200**	401, 403, 500
GET	`/api/v1/contato/{id}`	Busca uma mensagem pelo id	ADMIN	**200**	401, 403, 404, 500
PUT	`/api/v1/contato/{id}`	Atualiza o andamento (NOVO, EM_ANDAMENTO, RESOLVIDO)	ADMIN	**200**	400, 401, 403, 404, 500
DELETE	`/api/v1/contato/{id}`	Exclui uma mensagem	ADMIN	**204**	401, 403, 404, 500
Conteúdos					
GET	`/api/v1/conteudos`	Lista notícias (filtros ?categoria= e ?busca=)	Público	**200**	500
GET	`/api/v1/conteudos/{id}`	Busca um conteúdo pelo id	Público	**200**	404, 500
POST	`/api/v1/conteudos`	Cadastra um conteúdo	ADMIN	**201**	400, 401, 403, 500
PUT	`/api/v1/conteudos/{id}`	Atualiza um conteúdo	ADMIN	**200**	400, 401, 403, 404, 500
DELETE	`/api/v1/conteudos/{id}`	Exclui um conteúdo	ADMIN	**204**	401, 403, 404, 500

Total: 65 endpoints.

5. Exemplos de requisição e resposta

Cadastro – POST /api/v1/auth/cadastro

```
Requisição:
{ "nome": "Ana Souza", "email": "ana@email.com", "senha": "123456" }

Resposta 201 Created:
{
  "token": "9f2c...e41a",
  "usuario": { "id": 6, "nome": "Ana Souza", "email": "ana@email.com",
               "pontos": 0, "nivel": "Bronze", "recompensa": null }
}
```

Login – POST /api/v1/auth/login

```
Requisição:
{ "email": "ana@email.com", "senha": "123456" }

Resposta 200 OK: { "token": "...", "usuario": { ... } }
Senha errada – 401: { "mensagem": "E-mail ou senha incorretos." }
Após 5 erros seguidos – 429: { "mensagem": "Muitas tentativas de login. Tente novamente em
alguns minutos." }
```

Envio de ação com comprovante – POST /api/v1/acoes/enviar

```
curl -X POST http://localhost:8080/api/v1/acoes/enviar \
  -H "Authorization: Bearer <token>" \
  -F codigo=reciclagem-residuos \
  -F "descricao=Separei o lixo reciclavel da semana" \
  -F arquivo=@foto.jpg

Resposta 201 Created:
{ "idAcao": 12, "status": "APROVADA", "pontosGanhos": 100,
  "bonus": 0, "pontosTotais": 350, "nivel": "Prata" }

Vídeo com mais de 45 s – 400: { "mensagem": "O vídeo deve ter no máximo 45 segundos" }
```

Resumo do EcoRank – GET /api/v1/ecorank/me

```
Resposta 200 OK:
{ "idUsuario": 1, "nome": "InovaTech", "pontos": 250, "acoes": 2, "nivel": "Prata",
  "multiplicador": 1.2, "proximoNivel": "Ouro", "pontosNivelAtual": 200,
"pontosProximoNivel": 400,
  "pontosRestantes": 150, "progressoNivel": 25, "posicao": 3, "totalUsuarios": 5,
  "pontosSemana": 250, "acoesSemana": 2, "co2Kg": 2.4,
  "categorias": [ { "rotulo": "Reciclagem", "quantidade": 1, "percentual": 50 }, ... ] }
```

Erros de permissão

```
GET /api/v1/usuarios/me sem token  -> 401 { "mensagem": "Faça login para continuar." }
GET /api/v1/usuarios (usuário comum) -> 403 { "mensagem": "Acesso restrito a
administradores." }
GET /api/v1/objetivos/99 de outro usuário -> 404 { "mensagem": "Objetivo não encontrado com o
id 99" }
```

6. Protótipo – prints das telas implementadas

As telas abaixo pertencem ao front-end em React (plataforma SoulUp), que consome a API do EcoRank. Em cada tela está indicado qual endpoint fornece os dados.

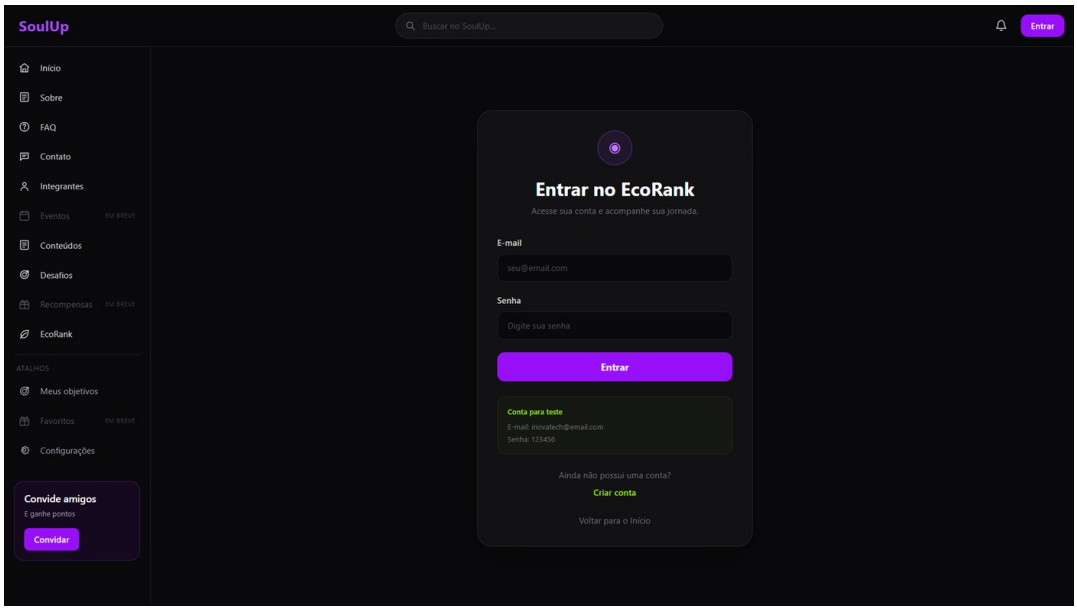


Figura 1 – Login: autentica em POST /api/v1/auth/login e mantém a sessão por token.

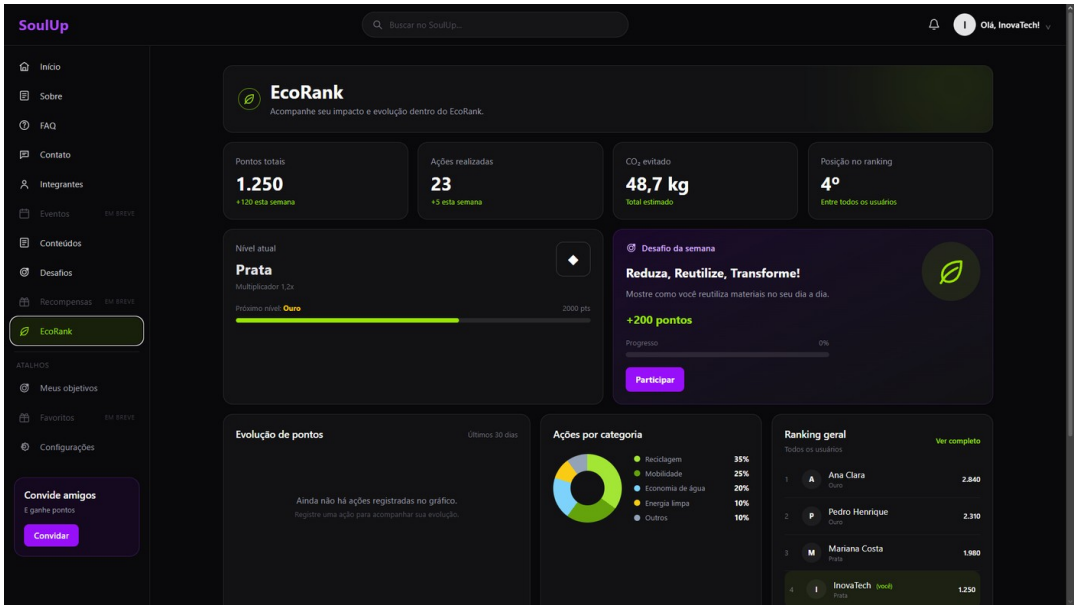


Figura 2 – Dashboard do EcoRank: pontos, nível, posição, impacto e ranking (GET /ecorank/me, /ranking e /impacto/me).

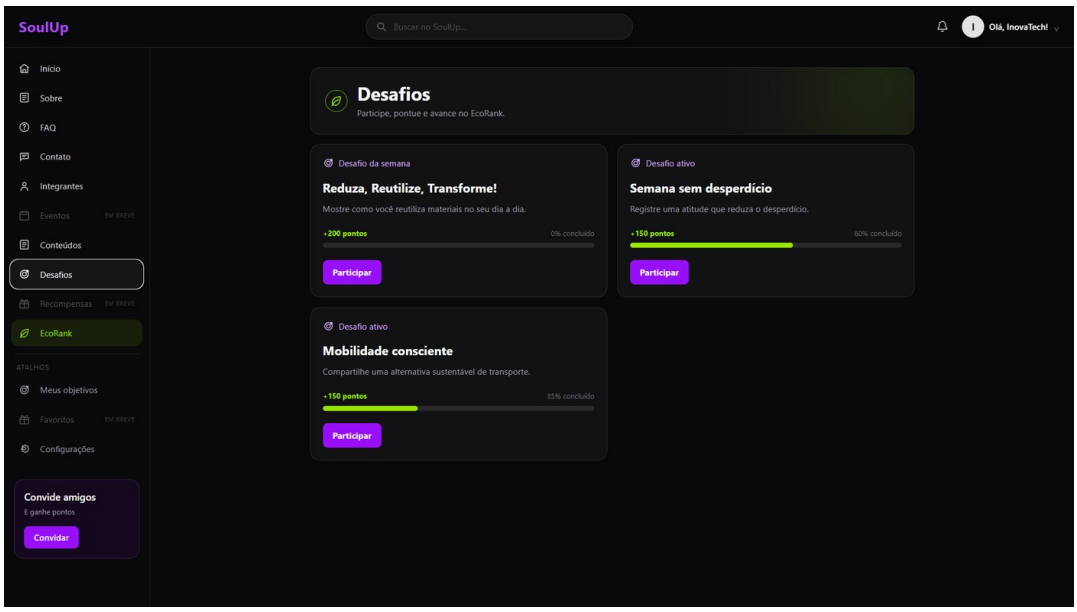


Figura 3 – Desafios: catálogo e progresso individual (GET /desafios/me; participar em POST /desafios/{id}/participar).

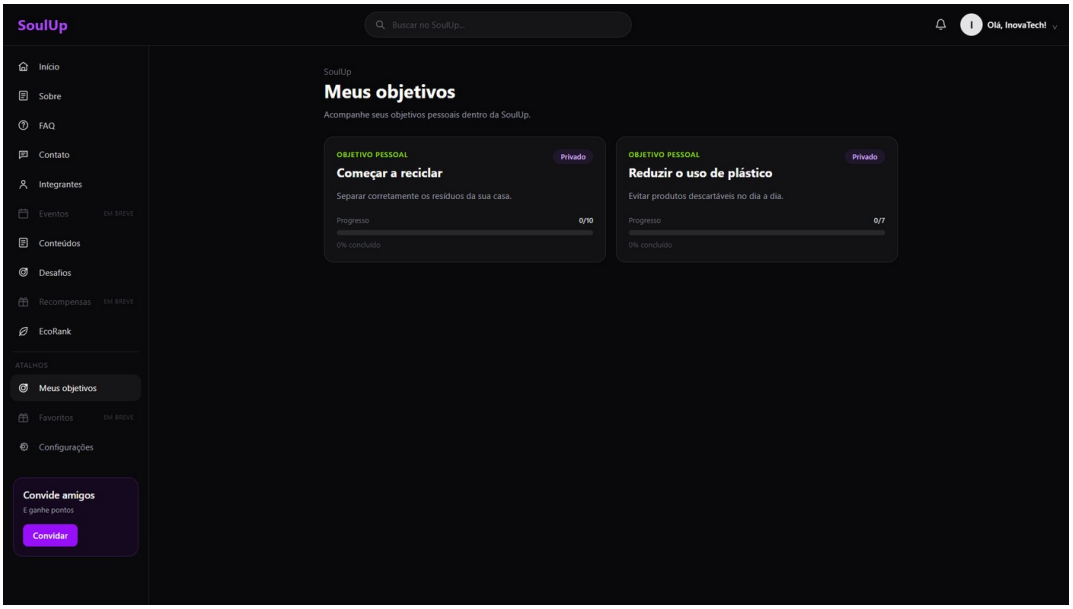


Figura 4 – Meus objetivos: objetivos pessoais do usuário (GET/POST/PUT/DELETE /objetivos).

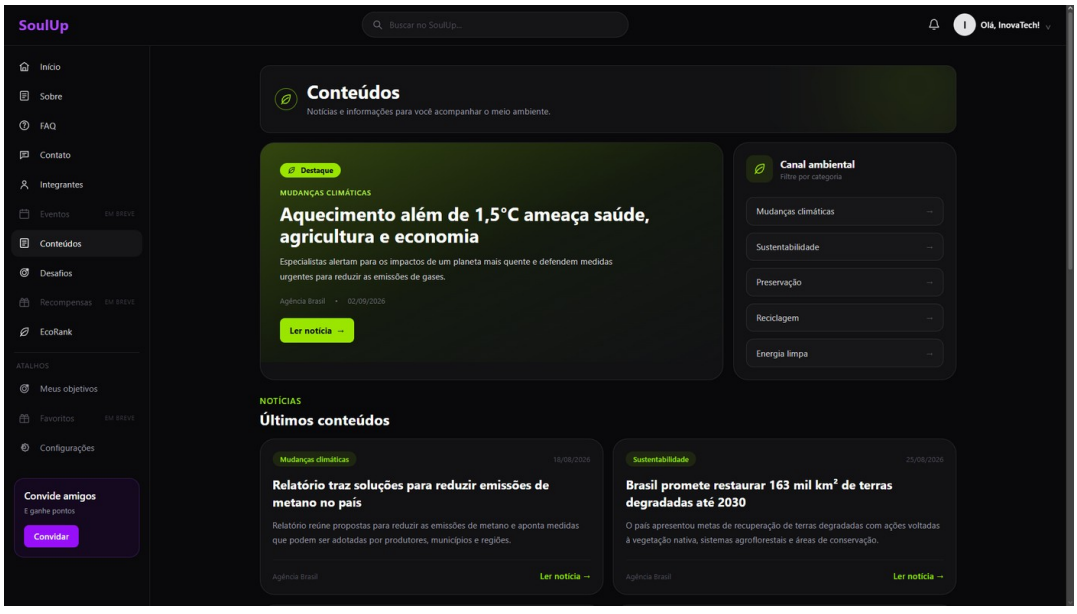


Figura 5 – Conteúdos: notícias ambientais filtradas por categoria (GET /conteudos).

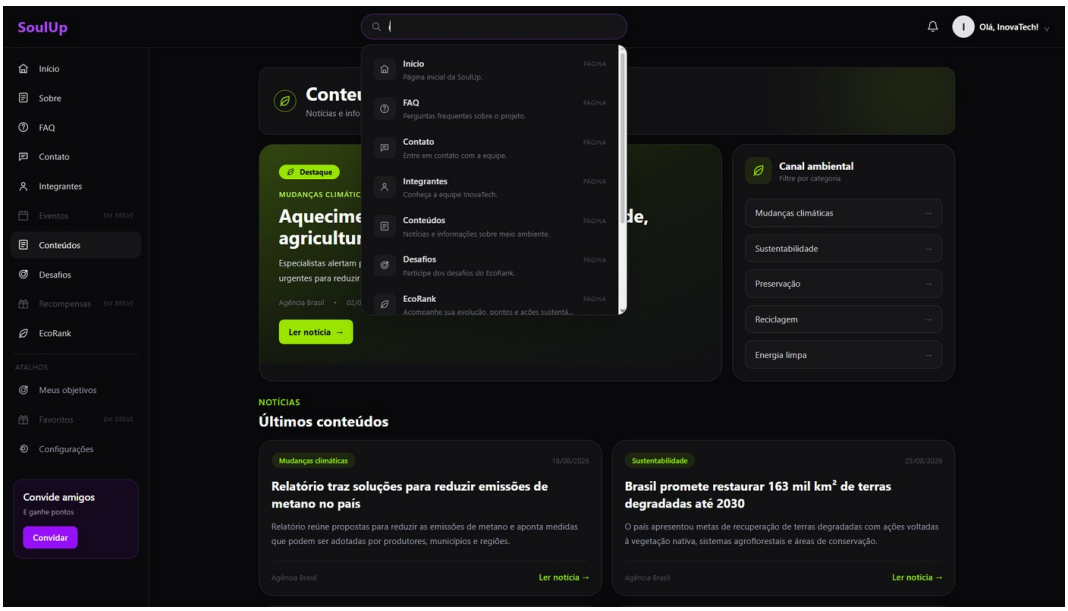


Figura 6 – Busca global no cabeçalho, que também consulta os conteúdos pela API.

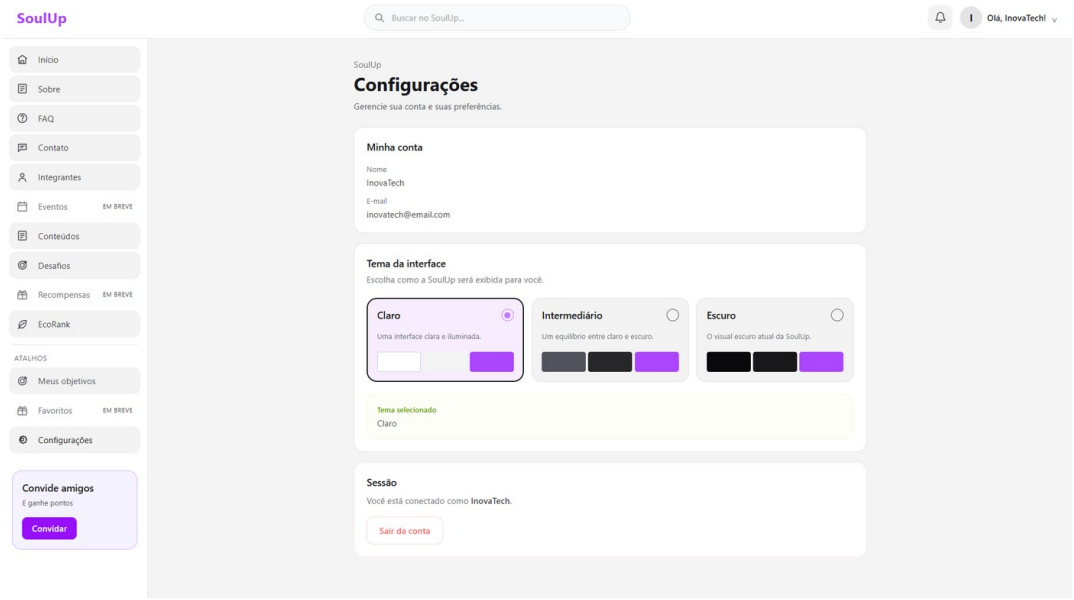


Figura 7 – Configurações (tema claro): dados da conta (GET/PUT /usuarios/me) e preferência de tema.

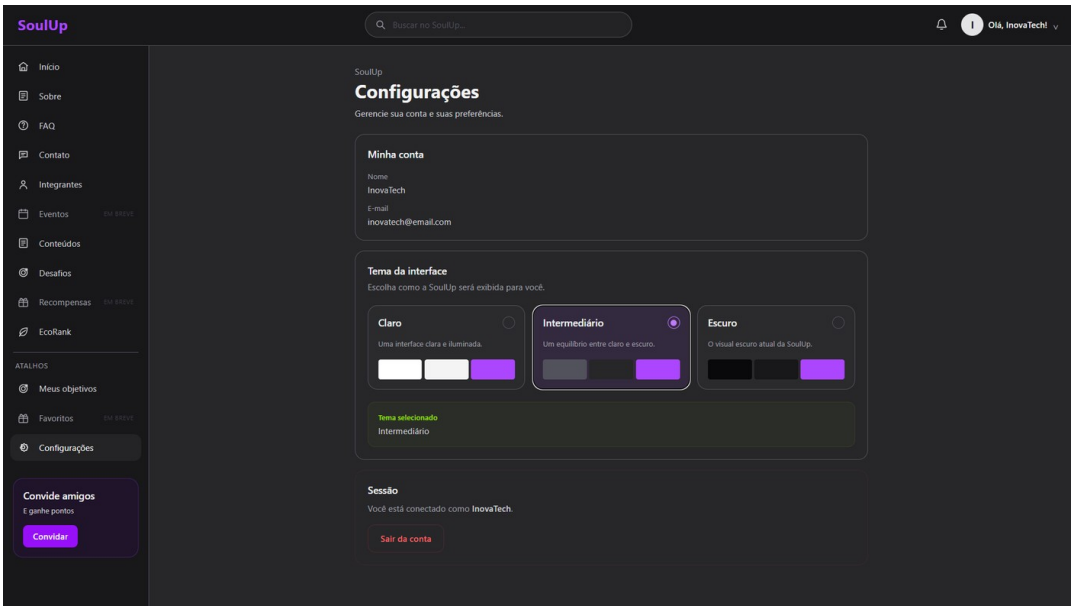


Figura 8 – Configurações (tema escuro).

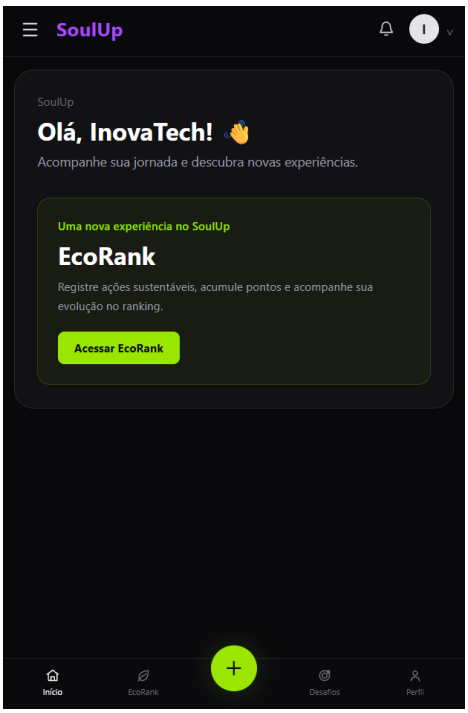


Figura 9 – Versão mobile da tela inicial, com acesso ao EcoRank.

7. Modelo de Entidade-Relacionamento (MER)

O banco Oracle possui 13 tabelas, criadas pelo arquivo script_banco.sql. O diagrama mostra as entidades, seus atributos, as chaves primárias (PK), estrangeiras (FK) e únicas (UK) e as cardinalidades dos relacionamentos (1 : N). A ligação tracejada entre USUARIO e NIVEL é uma regra de negócio (o nível é definido pelos pontos), sem chave estrangeira. Os tipos de dados de cada coluna estão no script_banco.sql e no resumo da próxima página.

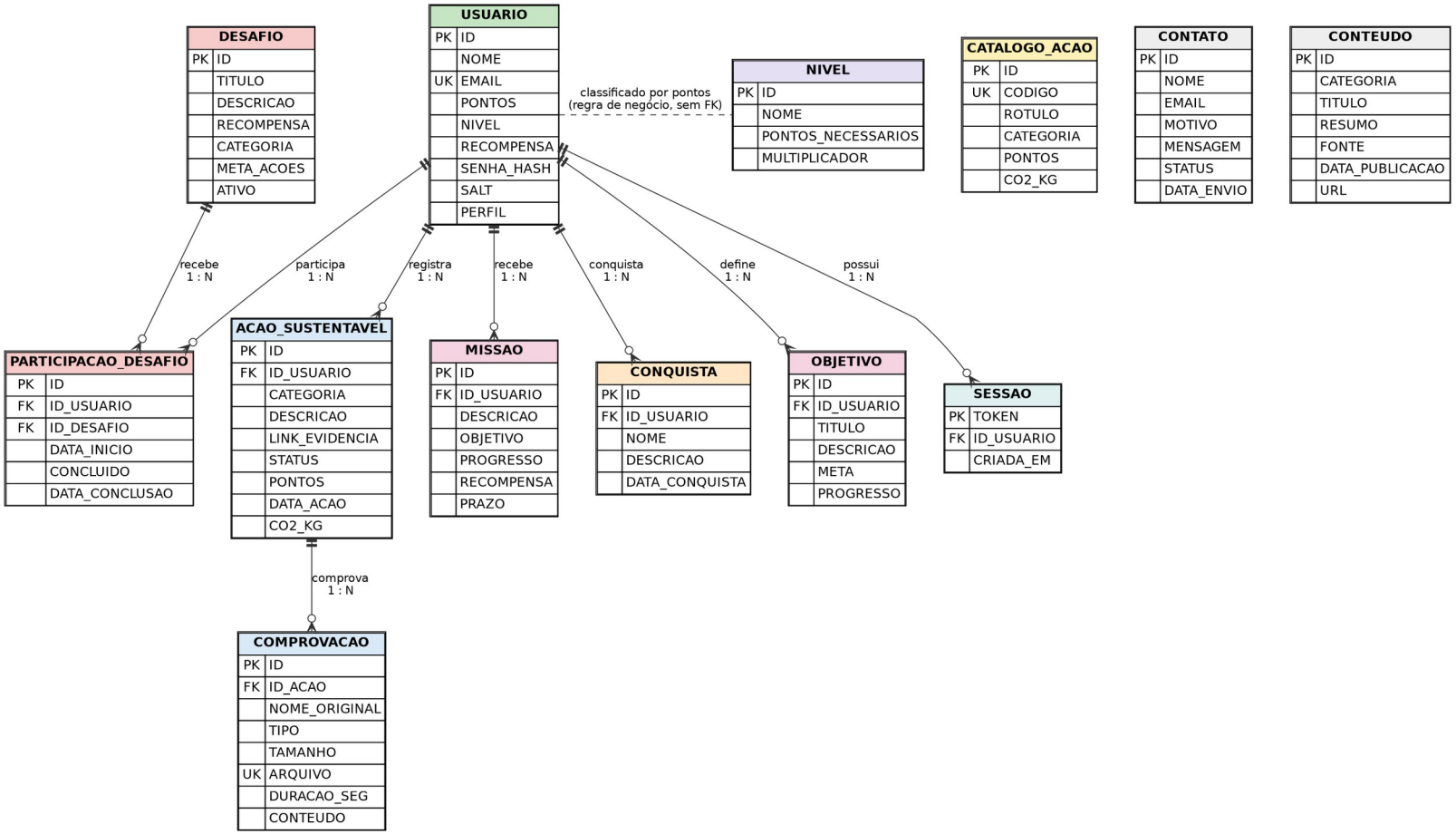


Figura 10 – MER do EcoRank (gerado a partir do script_banco.sql).

Resumo das tabelas e tipos de dados

Tabela	Função e principais colunas
USUARIO	Conta do usuário. ID (identity), NOME, EMAIL (único), PONTOS, NIVEL, RECOMPENSA, SENHA_HASH, SALT e PERFIL (USUARIO ou ADMIN).
SESSAO	Tokens de sessão ativos. TOKEN VARCHAR2(64) (PK), ID_USUARIO (FK), CRIADA_EM TIMESTAMP; validade de 7 dias.
NIVEL	Níveis do ranking. NOME VARCHAR2(30), PONTOS_NECESSARIOS NUMBER(10), MULTIPLICADOR NUMBER(4,2).
CATALOGO_ACAO	Ações oficiais. CODIGO (único), ROTULO, CATEGORIA, PONTOS NUMBER(10) e CO2_KG NUMBER(8,2) (estimativa).
ACAO_SUSTENTAVEL	Ações registradas. ID_USUARIO (FK), CATEGORIA, DESCRICAO VARCHAR2(300), LINK_EVIDENCIA, STATUS, PONTOS, DATA_ACAO DATE e CO2_KG.
COMPROVACAO	Foto ou vídeo de cada ação. ID_ACAO (FK), NOME_ORIGINAL, TIPO, TAMANHO, ARQUIVO (único), DURACAO_SEG e CONTEUDO BLOB.
DESAFIO	Desafios. TITULO, DESCRICAO, RECOMPENSA, CATEGORIA (opcional), META_ACOES e ATIVO.
PARTICIPACAO_DESAFIO	Participação do usuário em um desafio (único por par usuário/desafio). DATA_INICIO, CONCLUIDO, DATA_CONCLUSAO.
OBJETIVO	Objetivos pessoais. ID_USUARIO (FK), TITULO, DESCRICAO, META e PROGRESSO NUMBER(5).
MISSAO	Missões do usuário. ID_USUARIO (FK), DESCRICAO, OBJETIVO, PROGRESSO, RECOMPENSA e PRAZO DATE.
CONQUISTA	Conquistas desbloqueadas. ID_USUARIO (FK), NOME, DESCRICAO e DATA_CONQUISTA DATE.
CONTATO	Mensagens do “Fale conosco”. NOME, EMAIL, MOTIVO, MENSAGEM VARCHAR2(2000), STATUS e DATA_ENVIO.
CONTEUDO	Notícias ambientais. CATEGORIA, TITULO, RESUMO, FONTE, DATA_PUBLICACAO DATE e URL.

Todas as chaves estrangeiras usam ON DELETE CASCADE: ao excluir um usuário, seus dados são removidos.

8. Diagrama de classes atualizado

Os diagramas foram gerados a partir do código-fonte Java do projeto e seguem o padrão visto em aula: cada classe mostra seus atributos e métodos, e as relações aparecem como associação (1 : N), dependência (setas) e herança (seta com ponta vazada). Cada camada tem uma cor: Model (azul), Repository (rosa), Service (amarelo), Resource (verde), DTO (lilás), Mapper (laranja) e Exception (cinza). Nos services e resources, a linha “usa:” lista as classes das quais cada uma depende (injeção de dependência pelo construtor). Os métodos get e set do model aparecem resumidos como “getters e setters”.

8.1 Camada Model

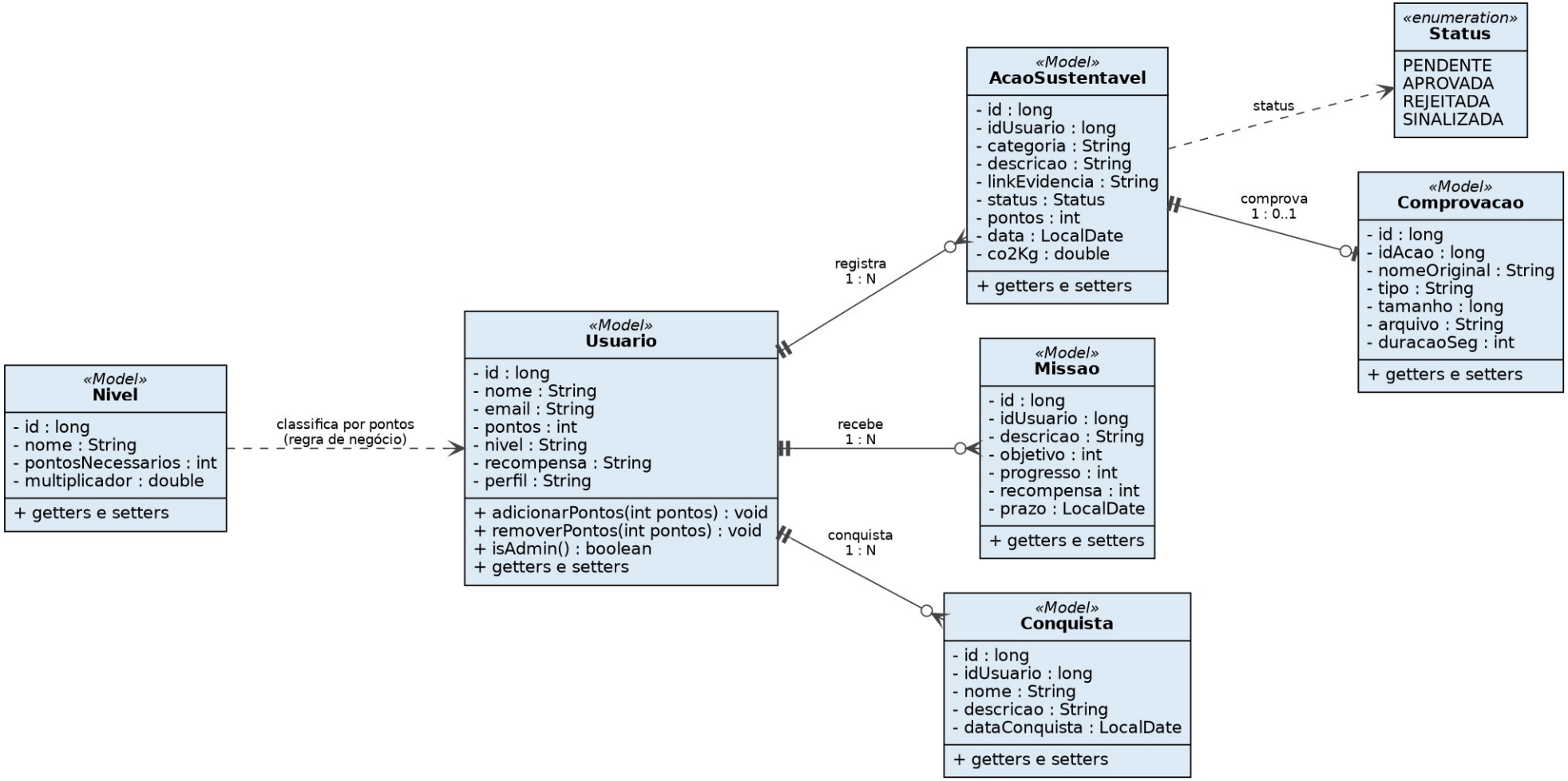


Figura 11 – Model: Usuario e suas associações com ações, missões e conquistas; Comprovacao e o enum Status.

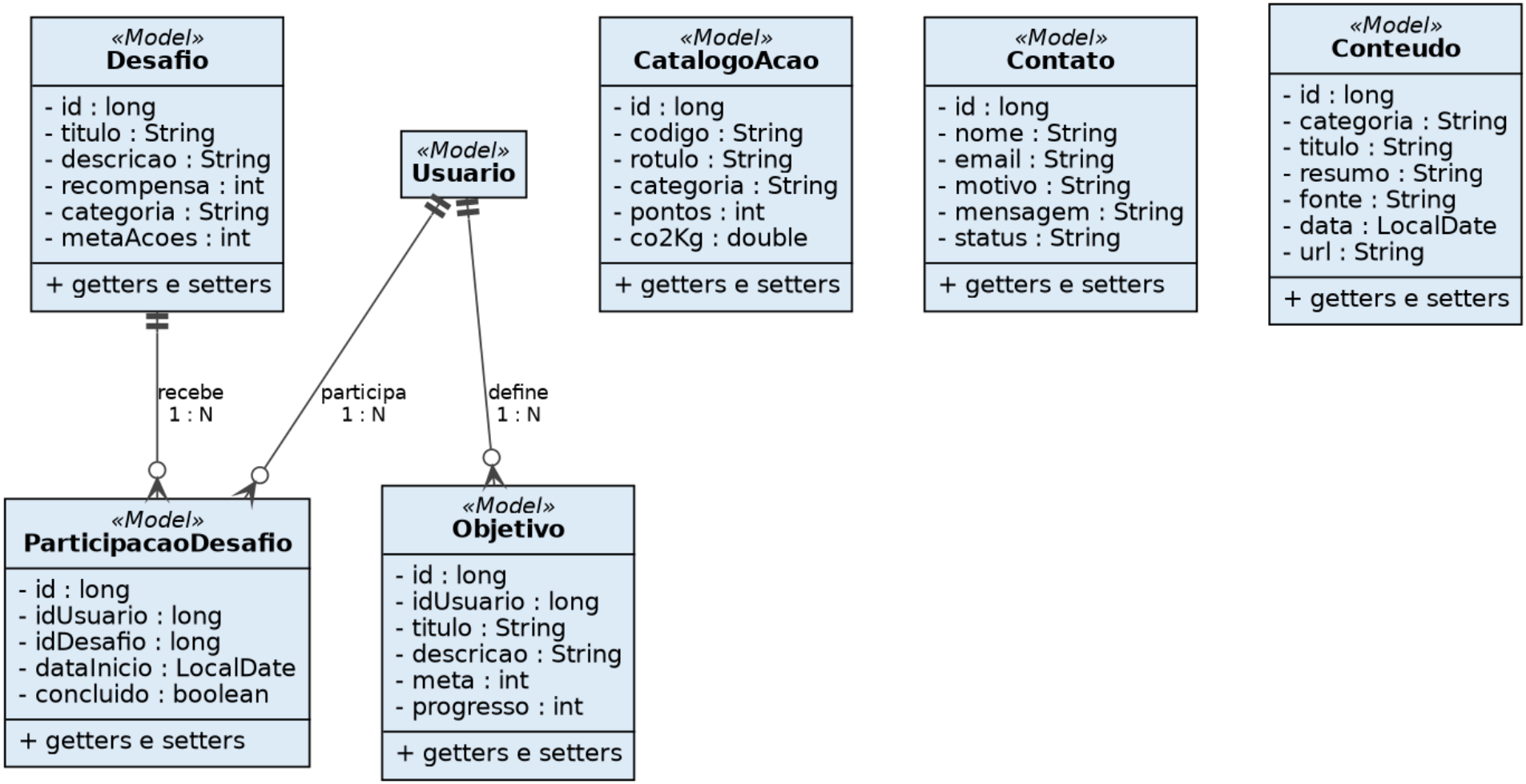


Figura 12 – Model: desafios, participações, objetivos, catálogo, contato e conteúdo.

8.2 Herança e exceções

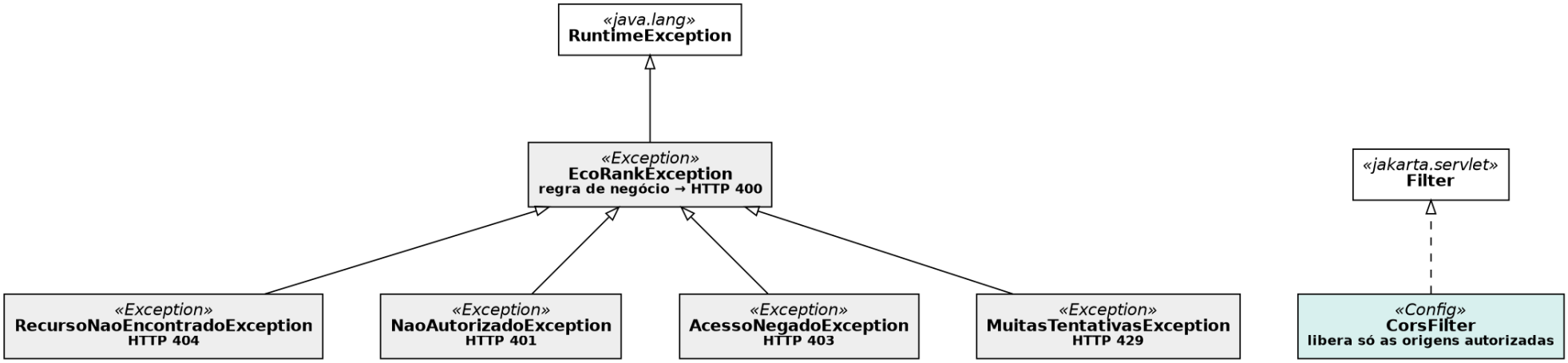


Figura 13 – Hierarquia das exceções (cada uma mapeada para um status HTTP) e CorsFilter, que implementa Filter.

8.3 Camada Repository (DAO)

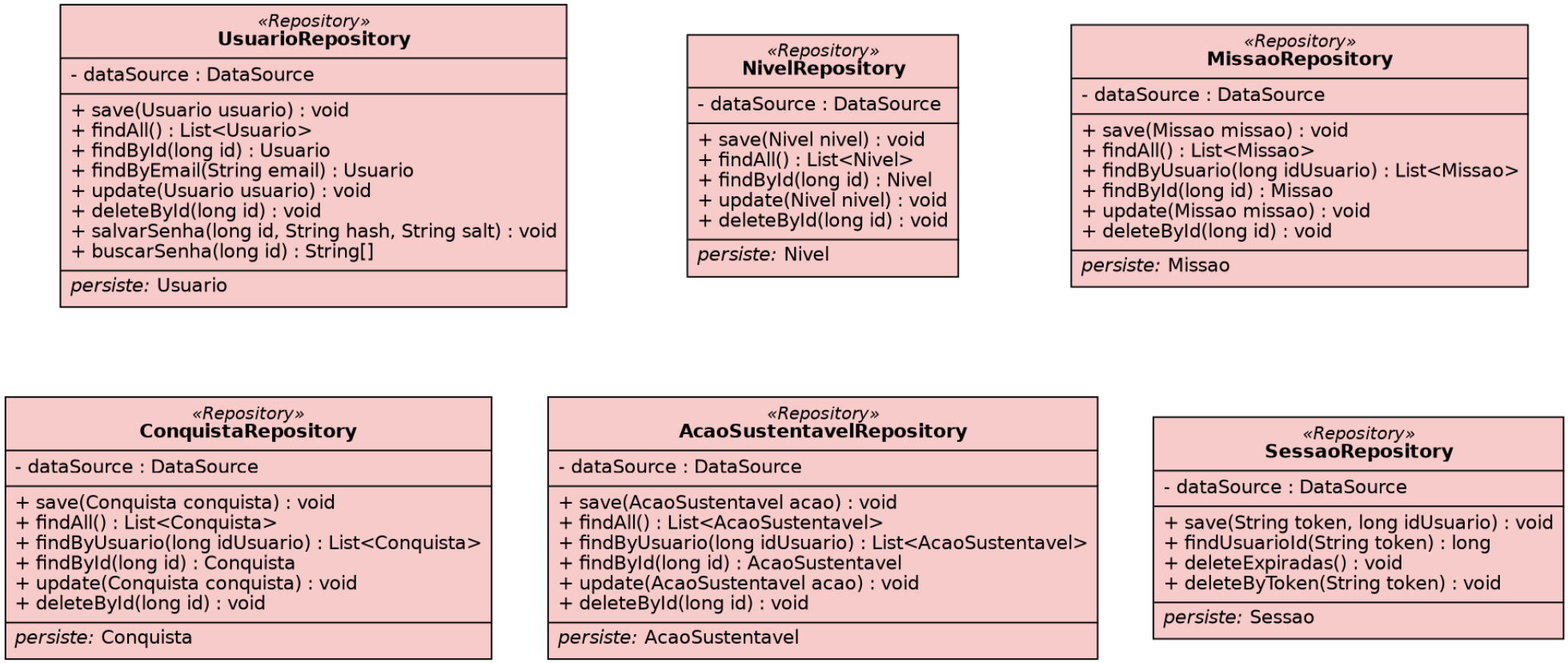


Figura 14 – Repositories (1/2): CRUD em JDBC com DataSource.

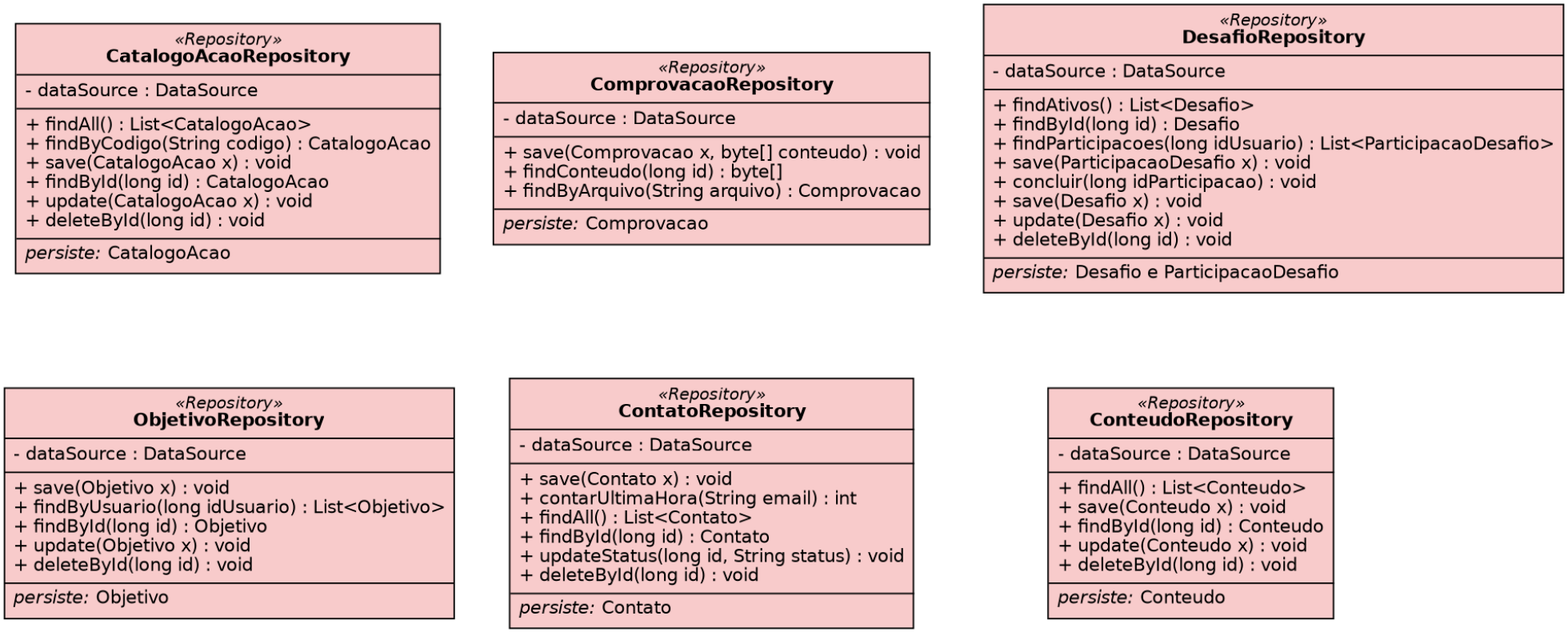


Figura 15 – Repositories (2/2).

8.4 Camada Service (BO)

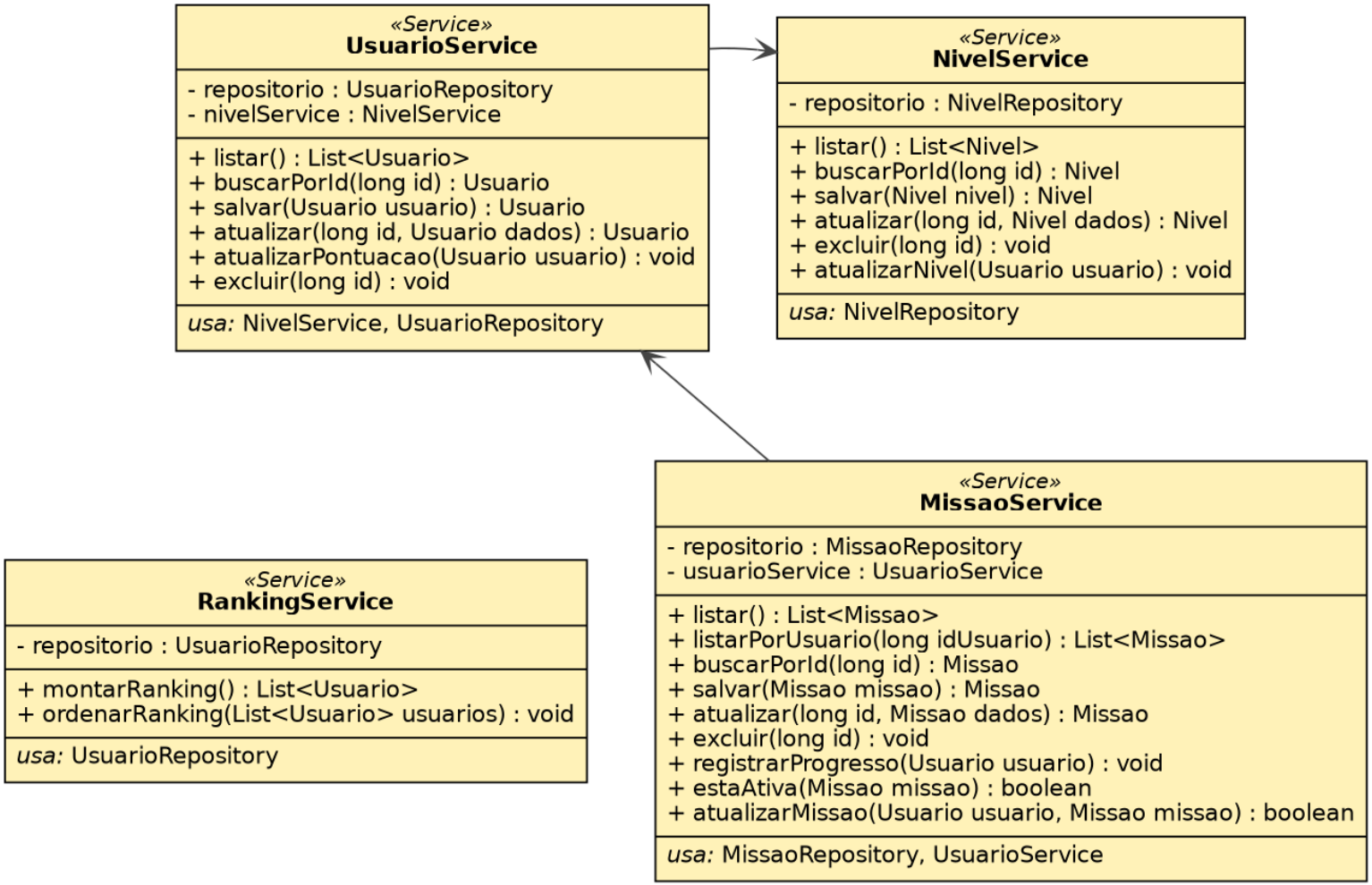


Figura 16 – Services (1/6): usuários, níveis, ranking e missões.

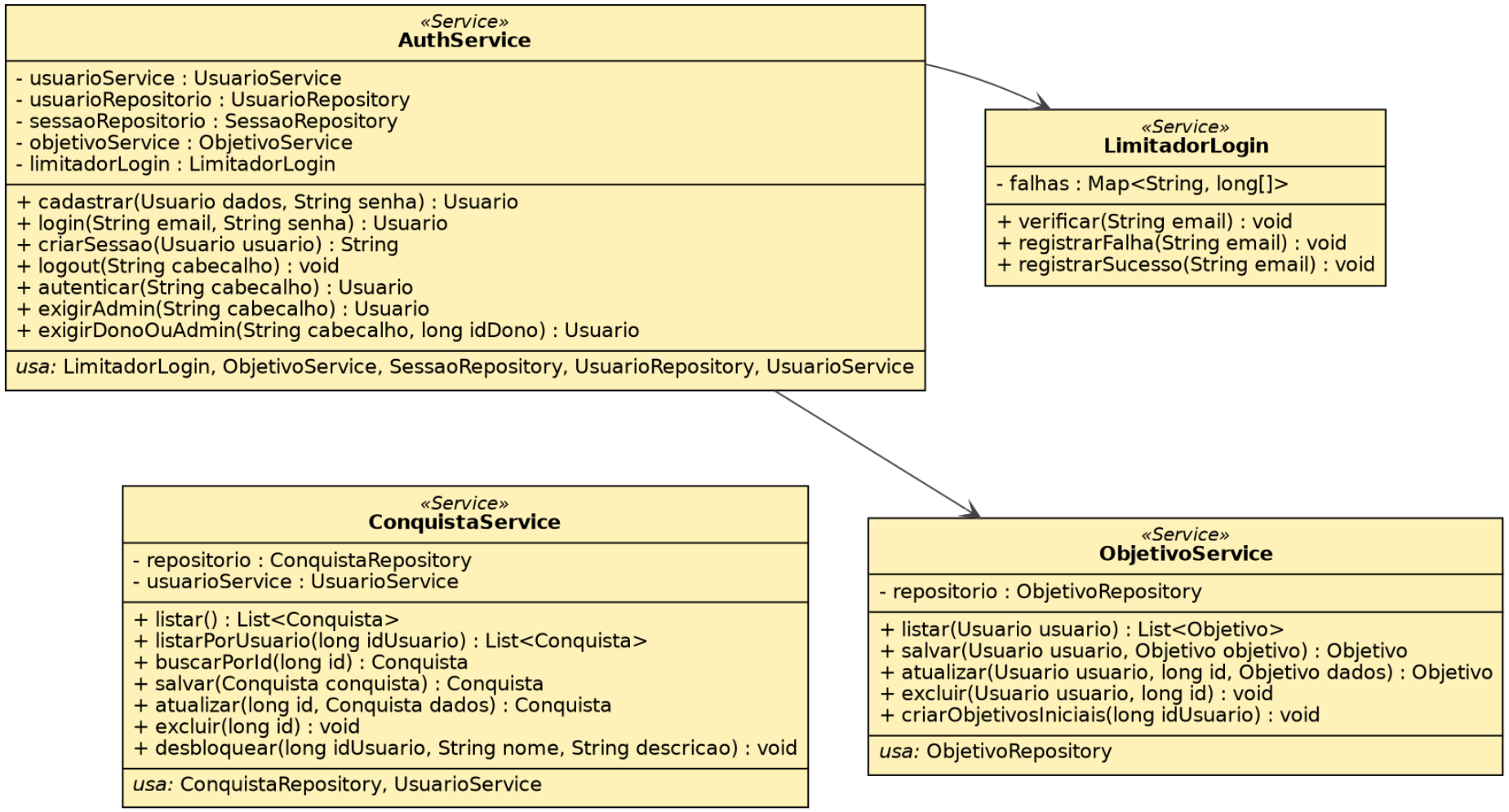


Figura 17 – Services (2/6): autenticação, limite de login, conquistas e objetivos.

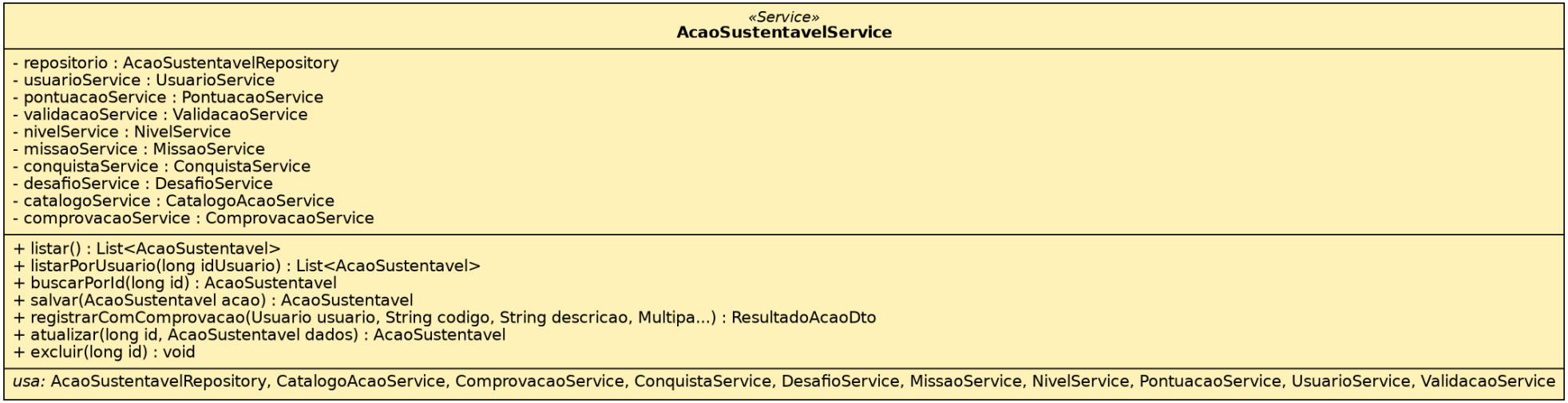


Figura 18 – Services (3/6): registro de ações (cálculo de pontos, níveis, missões e desafios).

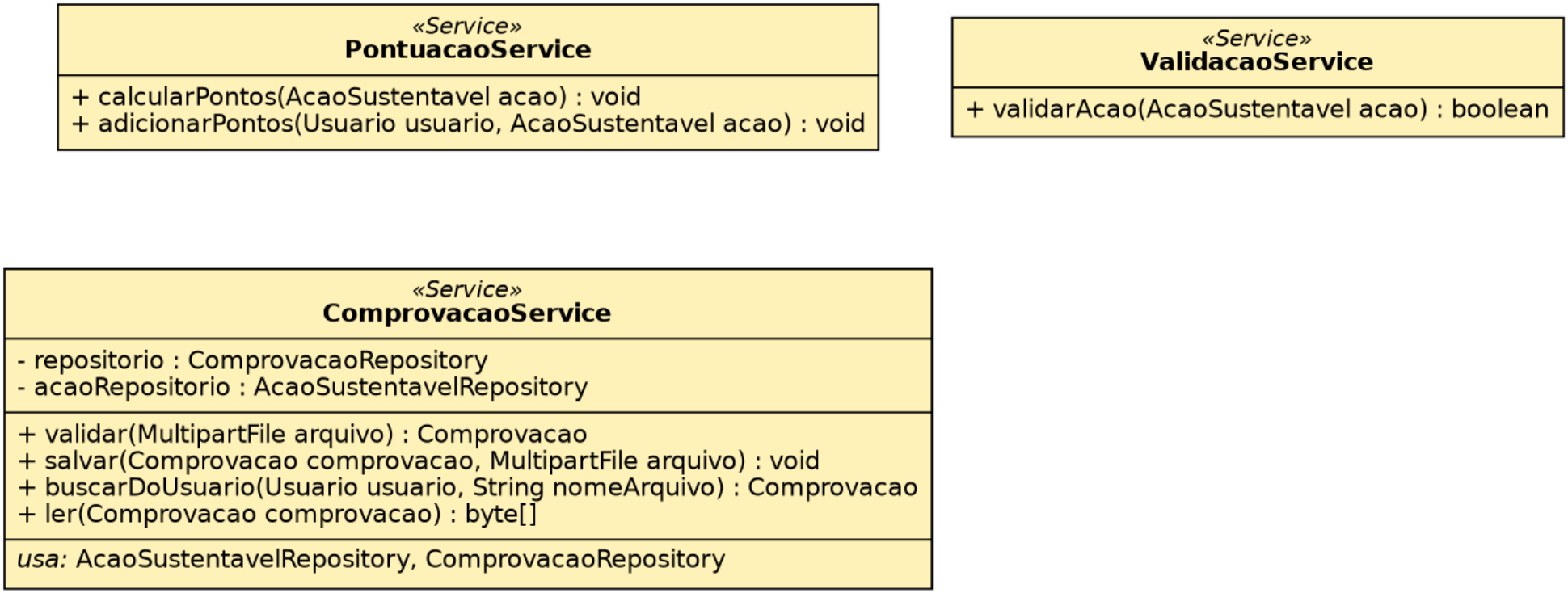


Figura 19 – Services (4/6): pontuação, validação e comprovação.

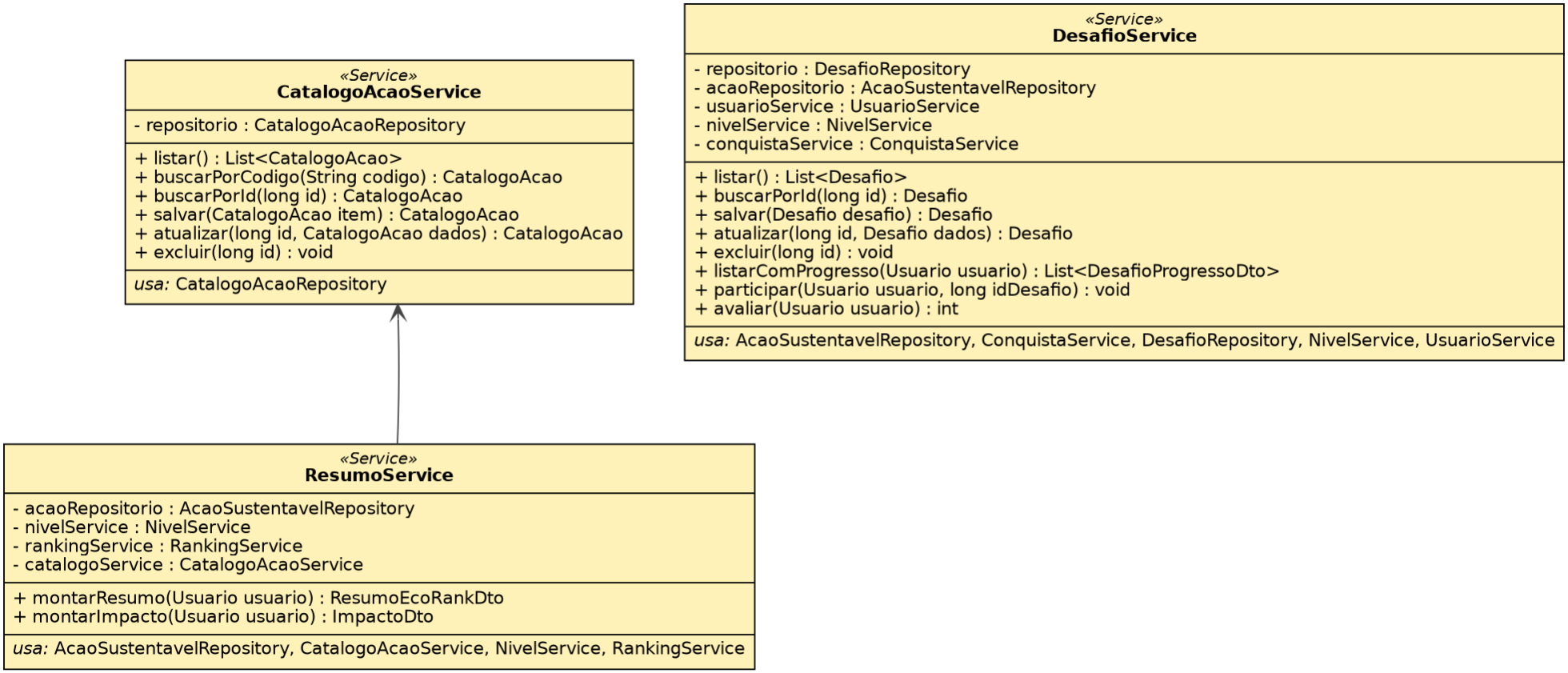


Figura 20 – Services (5/6): catálogo, desafios e resumo/impacto.

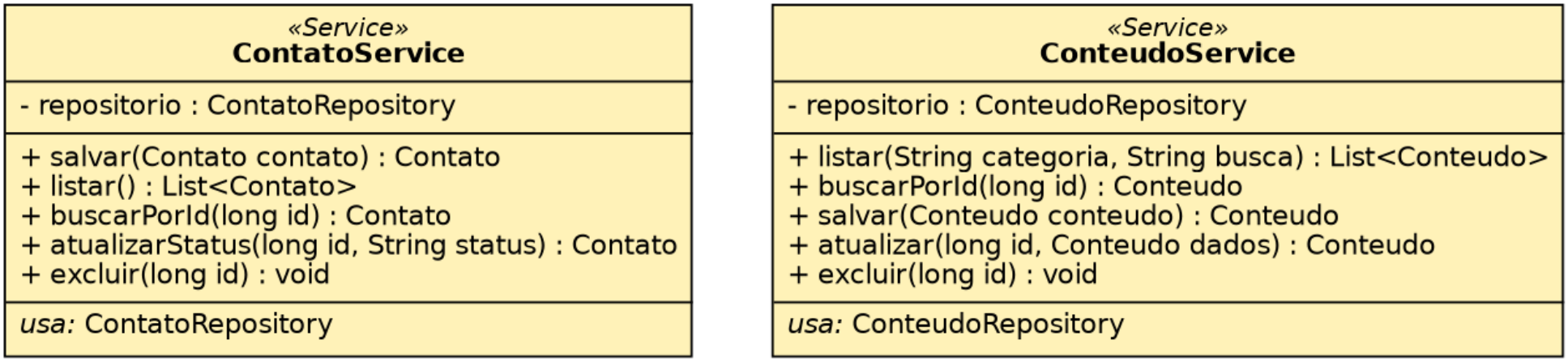


Figura 21 – Services (6/6): contato e conteúdos.

8.5 Camada Resource

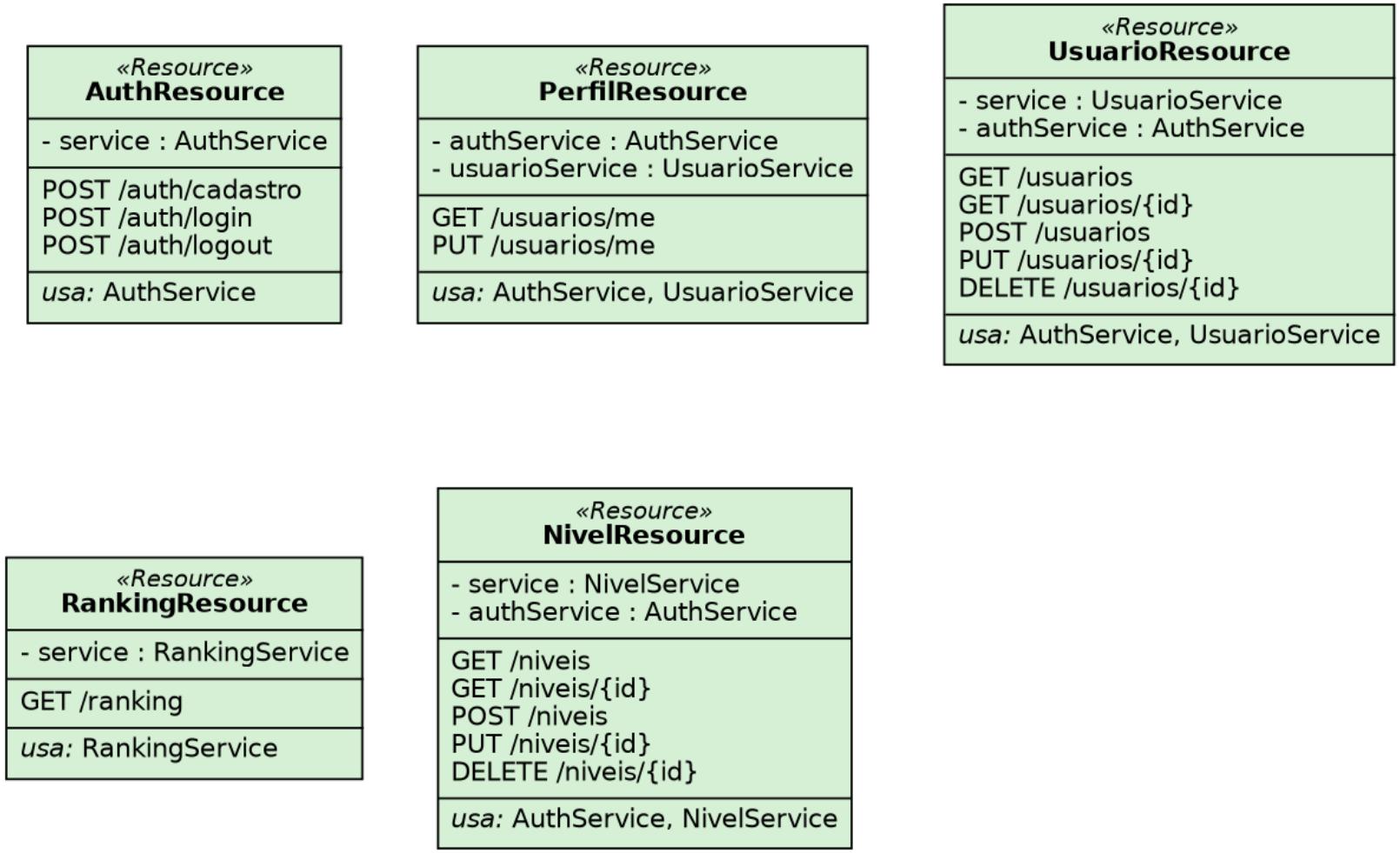


Figura 22 – Resources (1/3): autenticação, perfil, usuários, ranking e níveis.

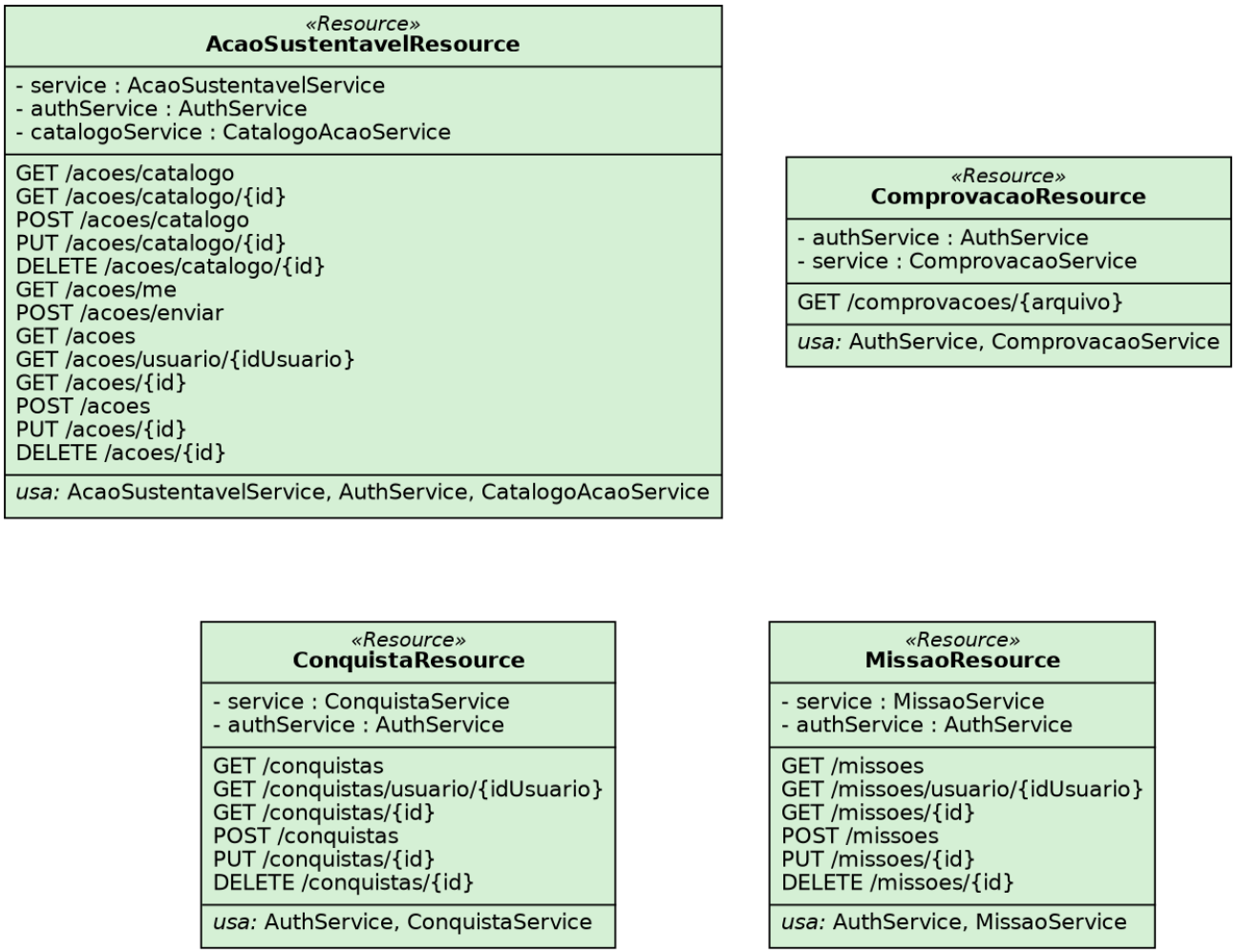


Figura 23 – Resources (2/3): ações, comprovações, conquistas e missões.

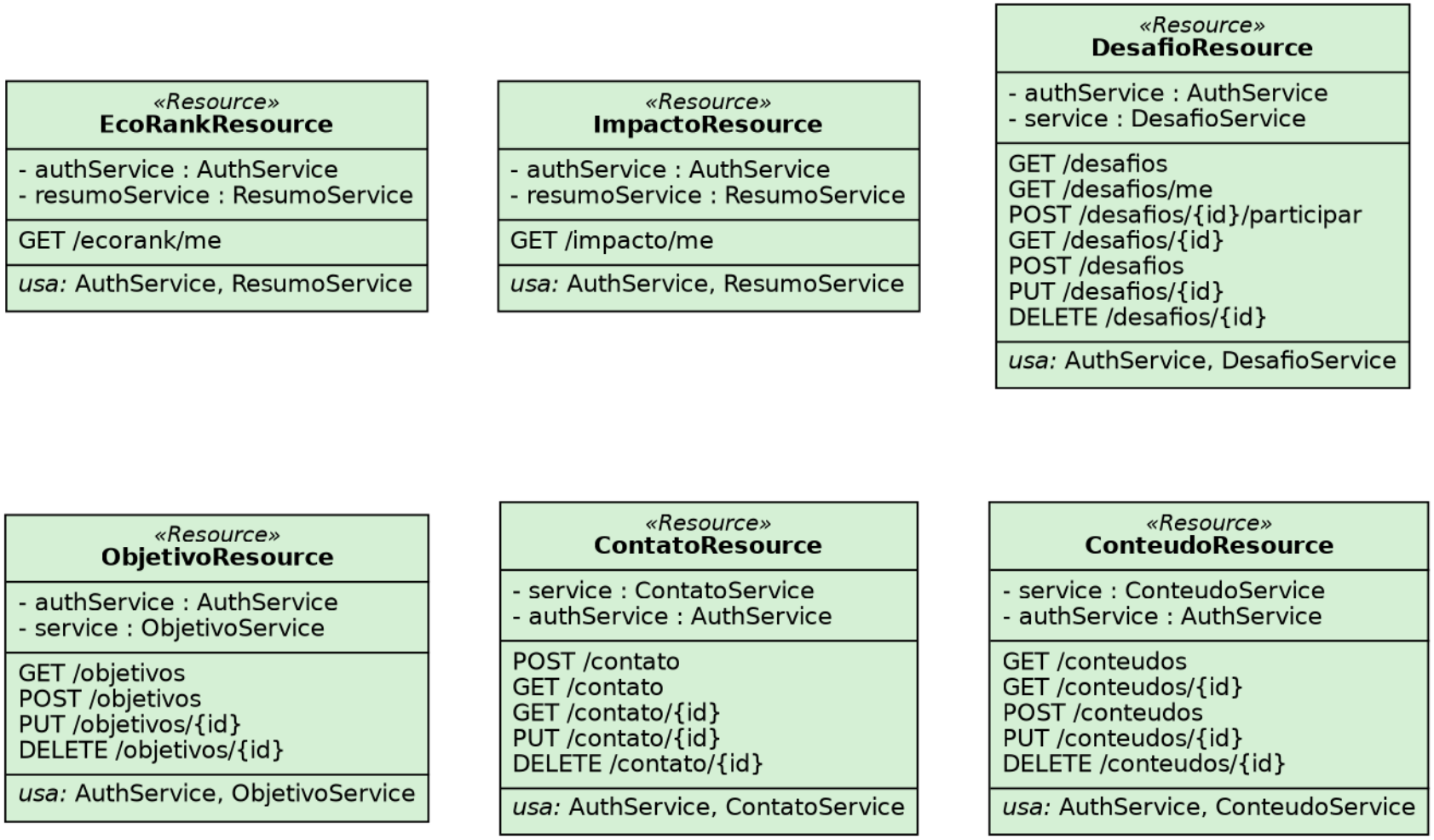


Figura 24 – Resources (3/3): EcoRank, impacto, desafios, objetivos, contato e conteúdos.

8.6 DTOs e Mappers

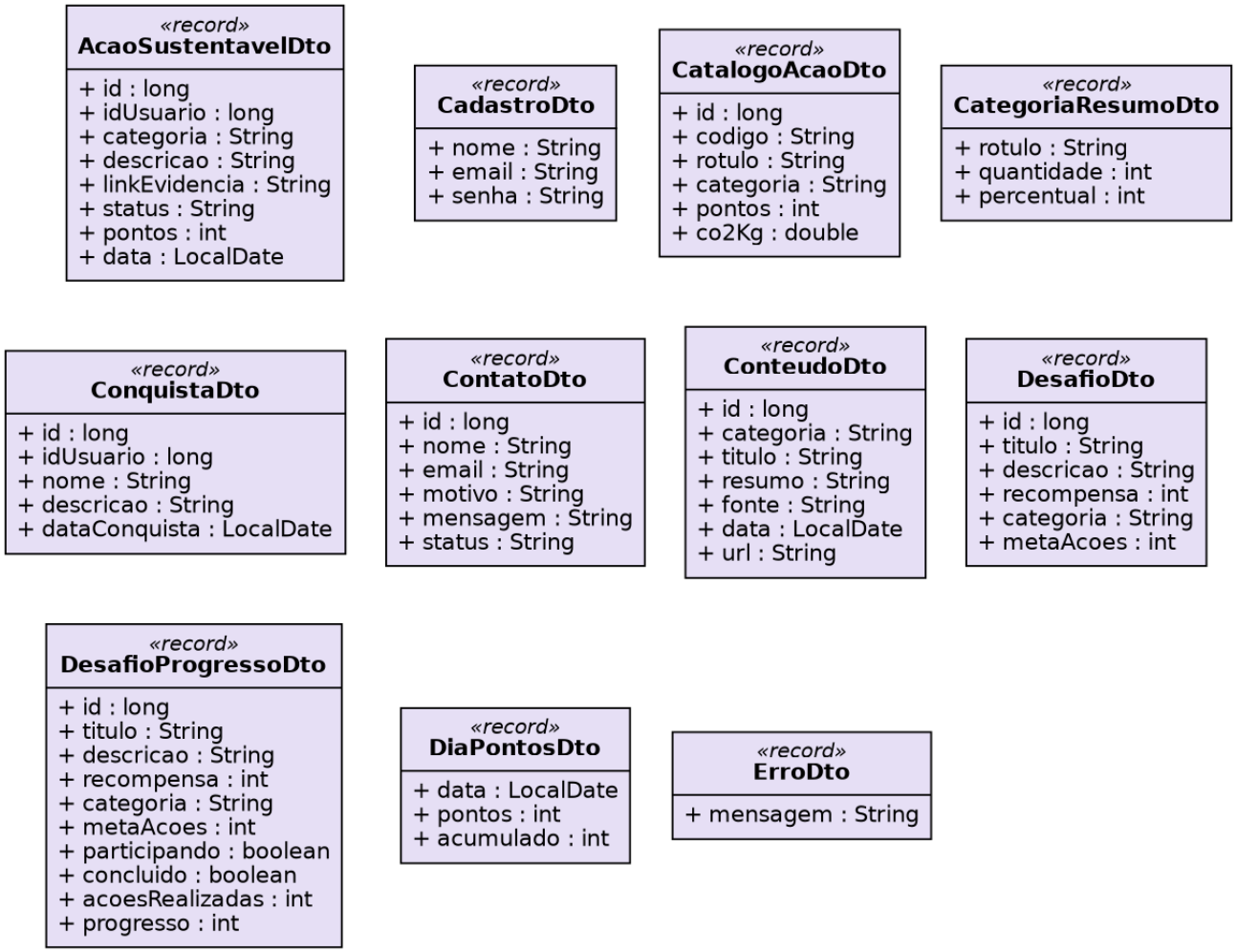


Figura 25 – DTOs (1/2): records que trafegam na API.

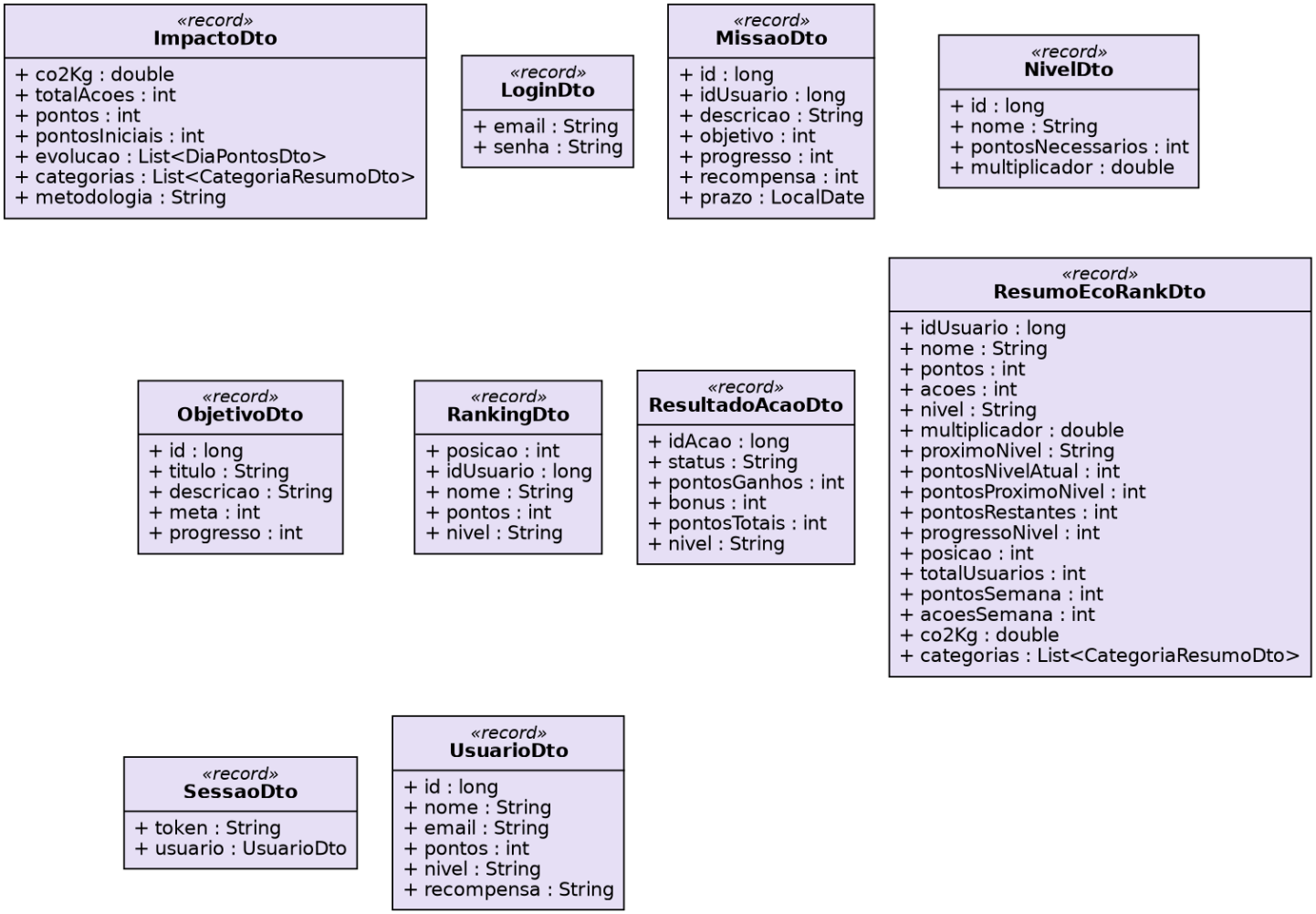


Figura 26 – DTOs (2/2).

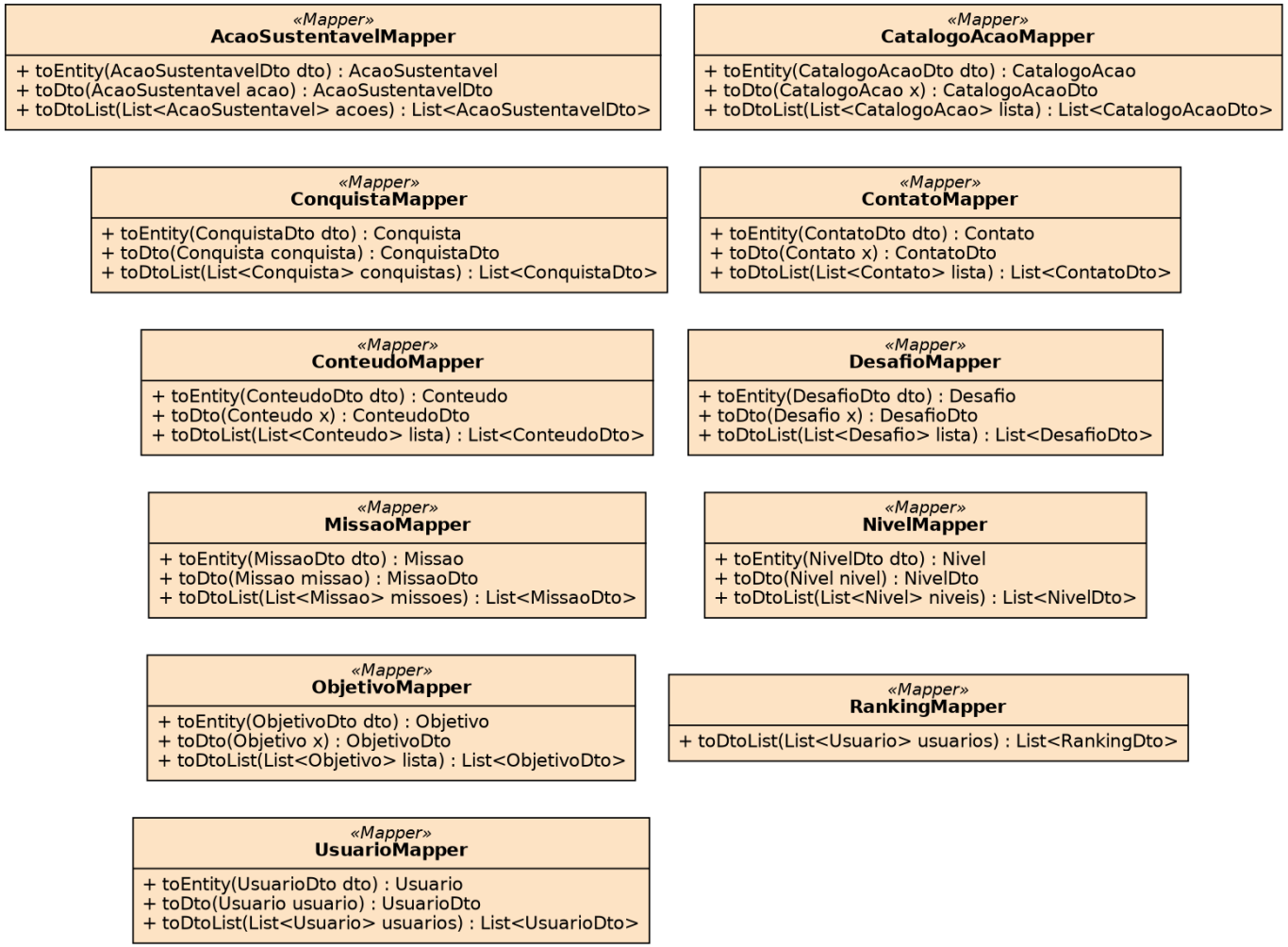


Figura 27 – Mappers: conversão entre DTO e entidade.

9. Procedimentos para rodar a aplicação

9.1 Ferramentas e versões

Ferramenta	Versão / uso
Java JDK	21 (obrigatório)
Maven	3.9 ou superior (ou o Maven embutido no IntelliJ IDEA)
IDE	IntelliJ IDEA (recomendada), Eclipse, NetBeans ou VS Code com extensões Java
Banco de dados	Oracle (usuário e senha da FIAP) e SQL Developer para executar o script
Teste da API	Postman, Insomnia ou curl
Front-end	Node.js 20 ou superior e npm (Vite + React)

9.2 Back-end (Java)

- **Importar o projeto:** extraia o ZIP e, no IntelliJ, use File > Open e selecione a pasta que contém o pom.xml. Aguarde o Maven baixar as dependências.
- **Criar o banco:** no SQL Developer, conecte no seu usuário e execute o script_banco.sql inteiro (arquivo em UTF-8). Ele recria as tabelas e cadastra dados de demonstração.
- **Variáveis de ambiente:** o projeto não guarda senha em arquivo. Defina DB_URL, DB_USERNAME e DB_PASSWORD (e CORS_ORIGINS em produção). No IntelliJ: Run > Edit Configurations > Environment variables.

```
DB_URL=jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl
DB_USERNAME=seu_usuario
DB_PASSWORD=sua_senha
```

- **Executar:** rode a classe EcorankApplication ou, pelo terminal:

```
mvn clean package -DskipTests
java -jar target/ecorank-0.0.1-SNAPSHOT.jar
```

- **Testar:** abra http://localhost:8080/api/v1/ranking no navegador; deve listar os usuários em JSON. Usuários de demonstração (senha 123456): inovatech@email.com, anaclara@email.com, pedrohenrique@email.com, mariana@email.com e joao@email.com.
- **Criar um administrador:** cadastre a conta pelo site e execute no Oracle: UPDATE USUARIO SET PERFIL = 'ADMIN' WHERE EMAIL = 'seu@email.com'; COMMIT;
- **Roteiro de testes:** o arquivo docs/ROTEIRO_TESTES.md traz 51 testes (cadastro, login, token expirado, permissões, upload, ranking, desafios e CORS).

9.3 Front-end (React)

```
npm install
copie .env.example para .env e ajuste VITE_API_URL (ex.: http://localhost:8080)
npm run dev      # abre em http://localhost:5173
```

10. Deploy da aplicação na nuvem (Render)

A aplicação é empacotada com o Dockerfile do projeto (build em duas etapas: Maven para gerar o .jar e uma imagem Java 21 enxuta para executar). O banco continua sendo o Oracle da FIAP.

- No GitHub, o repositório deve ter o pom.xml e o Dockerfile na raiz.
- No Render: New + > Web Service, conectar o repositório, Language = Docker, Dockerfile Path = ./Dockerfile.
- Em Environment Variables, cadastrar DB_URL, DB_USERNAME, DB_PASSWORD e CORS_ORIGINS (endereço exato do front-end publicado, sem barra no final).
- Aguardar o build e o status Live; testar a URL /api/v1/ranking.
- No front-end publicado, definir VITE_API_URL com a URL do Render e gerar o build novamente.

Observação: no plano gratuito do Render, o serviço “dorme” sem uso e a primeira chamada pode levar cerca de um minuto. Os comprovantes ficam no Oracle (coluna BLOB), por isso não se perdem quando o serviço reinicia.

11. Links da entrega

Item	Endereço
Repositório no GitHub	**(preencher antes de enviar)**
Deploy da API (Render)	**(preencher antes de enviar)**
Front-end publicado (opcional)	**(preencher antes de enviar)**

Entregáveis: código-fonte Java atualizado (pasta do projeto em .zip), esta documentação em PDF e o link do deploy na nuvem.